

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I K.AISHWARYA(192572116) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

A smart home automation system needs to make decisions based on **sensor inputs, user preferences, and environmental conditions**. Different types of intelligent agents can be used.

1. Simple Reflex Agent

A **simple reflex agent** selects an action based only on the **current percept** using condition-action rules.

Example:

- If motion is detected → **Turn ON light**
- If temperature > 28°C → **Turn ON AC**
- If it is night and motion is detected → **Turn ON security light**

Advantages

- Simple to design
- Fast response
- Requires less computation

Disadvantages

- Does not remember previous situations
- Cannot learn user preferences
- Cannot handle situations that require past information

Example in smart home:

If the motion sensor detects a person, the light immediately turns on.

2. Model-Based Agent

A **model-based agent** maintains an **internal model/state of the environment**. It uses both current and previous information to make decisions.

Example:

Suppose the system knows:

- The user normally sleeps at 11 PM.
- The bedroom light was turned off.

- The temperature has been increasing.

It can use this information to decide whether to adjust the AC or lighting.

Advantages

- Can remember previous conditions
- Works better in partially observable environments
- Can understand changes in the environment

Disadvantages

- More complex than a simple reflex agent
- Requires memory and an internal model

3. Goal-Based Agent

A **goal-based agent** chooses actions that help achieve a specific goal.

Possible smart-home goals:

- Maintain comfortable temperature
- Save electricity
- Keep the house secure
- Turn off unnecessary appliances

Example:

Goal: Save energy.

The agent may decide:

No person detected for 30 minutes → Turn off lights and AC.

The agent chooses the action based on whether it helps achieve the goal.

Advantages

- Can plan actions
- More flexible
- Suitable for achieving specific objectives

Disadvantages

- Requires more computation
- May not distinguish between two equally successful actions

4. Utility-Based Agent

A **utility-based agent** chooses the action that provides the **highest overall utility**.

It can consider several factors simultaneously:

- Comfort
- Energy consumption
- Security
- Cost
- User preferences

Example:

Suppose the temperature is high.

The agent could:

- Set AC to 22°C → high comfort but high energy consumption
- Set AC to 25°C → good comfort and lower energy consumption

A utility-based agent can choose 25°C because it provides a better balance between comfort and energy efficiency.

Advantages

- Handles multiple objectives
- Makes better decisions under trade-offs
- Suitable for complex environments

Disadvantages

- Utility function is more difficult to design
- Requires more computation

Comparison

Agent	Main Idea	Smart Home Example
Simple Reflex	Current condition → action	Motion → Light ON
Model-Based	Uses current + previous information	Remembers occupancy
Goal-Based	Works toward a goal	Reduce energy consumption
Utility-Based	Selects best overall option	Balance comfort + energy + security

Most Effective Approach

A **hybrid approach combining model-based, goal-based, utility-based, and learning capabilities** would be most effective.

Why?

Model-based:

Maintains information about the home's current state and previous conditions.

Goal-based:

Helps achieve goals such as comfort, energy saving, and security.

Utility-based:

Balances competing requirements. For example, it can maintain comfortable temperature while minimizing electricity consumption.

Learning:

Learns the user's habits over time.

Example

If the user normally comes home at 6 PM, the system can learn this pattern and:

Predict arrival → adjust temperature → turn on appropriate lights → maintain security.

Conclusion

A hybrid intelligent agent is the most suitable for a smart home because it can use current and past information, achieve specific goals, balance comfort, energy efficiency and security, and learn user preferences over time.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.
Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

A robot placed in an unknown maze needs to search for an exit without knowing the complete maze layout beforehand. **Uninformed search algorithms** can explore the maze without using additional domain-specific knowledge.

A. Uniform Cost Search (UCS)

Uniform Cost Search expands the node having the **lowest cumulative path cost**.

It uses a **priority queue**.

Working

1. Start from the robot's initial position.
2. Add it to the priority queue with cost 0.
3. Select the node with the lowest path cost.
4. Expand that node.
5. Add its unexplored neighbors with their cumulative costs.
6. Continue selecting the lowest-cost node.
7. Stop when the exit is reached.

Example

If:

$$S \rightarrow A = 2$$

$$S \rightarrow B = 1$$

$B \rightarrow G = 5$
 $A \rightarrow G = 2$

Then:

- $S \rightarrow B \rightarrow G = 1 + 5 = 6$
- $S \rightarrow A \rightarrow G = 2 + 2 = 4$

UCS selects:

$S \rightarrow A \rightarrow G$

because its total cost is **4**.

UCS Complexity

For branching factor **b**, optimal solution cost **C***, and minimum step cost **ε**:

- **Time:** approximately $O(bC^*/\epsilon)O(b^{\lceil C^*/\epsilon \rceil})$
- **Space:** approximately $O(bC^*/\epsilon)O(b^{\lceil C^*/\epsilon \rceil})$

Advantages

- Complete when step costs are positive.
- Optimal for finding the least-cost path.
- Useful when different movements have different costs.

Disadvantages

- Can use large amounts of memory.
- Can be slow when there are many possible paths.
- Maintains a large priority queue.

B. Iterative Deepening Search (IDS)

Iterative Deepening Search repeatedly performs depth-limited DFS.

It starts with:

Depth limit = 0

Then:

Depth limit = 1

Depth limit = 2

Depth limit = 3

...

until the goal is found.

Example

Depth 0 → Check S

Depth 1 → Check nodes one level away

Depth 2 → Check nodes up to depth 2

Depth 3 → Check nodes up to depth 3

...

IDS Complexity

For branching factor **b** and solution depth **d**:

- **Time:** $O(bd)O(b^d)$
- **Space:** $O(bd)O(bd)$

Advantages

- Requires much less memory than BFS.
- Complete when the branching factor is finite.
- Finds the shallowest solution when all actions have equal cost.

Disadvantages

- Repeats the search at every depth.
- Can perform the same node expansions multiple times.
- Not suitable for different path costs because it considers depth rather than total cost.

UCS vs IDS

Feature	UCS	IDS
Search type	Uninformed	Uninformed
Selection	Lowest path cost	Increasing depth limit
Data structure	Priority queue	DFS stack/recursion
Time	$O(bC*/\epsilon)O(b^{\{C^*/\epsilon\}})$	$O(bd)O(b^d)$
Space	$O(bC*/\epsilon)O(b^{\{C^*/\epsilon\}})$	$O(bd)O(bd)$
Complete	Yes, with positive costs	Yes, finite branching
Optimal	Yes, for positive costs	Yes only with equal step costs
Memory usage	High	Low
Best use	Different path costs	Limited memory + equal costs

C. Relation to Intelligent Agents

The robot can be designed using different intelligent-agent architectures.

Simple Reflex Agent

The robot reacts only to the current situation.

Example:

Obstacle ahead $\rightarrow$ Turn right.

Problem: It does not remember previously explored paths.

Model-Based Agent

The robot maintains an internal representation of the maze.

Example:

"I have already visited this cell and this path contains an obstacle."

This is more suitable for maze exploration.

Goal-Based Agent

The robot has a goal:

Reach the maze exit.

It chooses actions that move it toward that goal.

Utility-Based Agent

The robot considers the quality of different paths.

For example:

- Shorter path = better
- Lower energy consumption = better
- Safer route = better

It selects the path with the highest utility.

Most Effective Combination

For an unknown maze, a **model-based + goal-based agent using IDS or UCS** is appropriate.

If **all movements have equal cost**, I would choose:

Model-based Goal-Based Agent + IDS

because IDS uses very little memory and can find the shallowest solution.

If **different paths have different costs**, I would choose:

Model-based Goal-Based Agent + UCS

because UCS finds the **least-cost path**.

